



汇编程序设计课程实验报告

题目 综合程序设计实验 2

姓 名：王子琪

(学 号)：22920182204309

院 系：信息学院

专 业：人工智能

年 级：2018 级

2020 年 4 月 23 日

一、实验目的

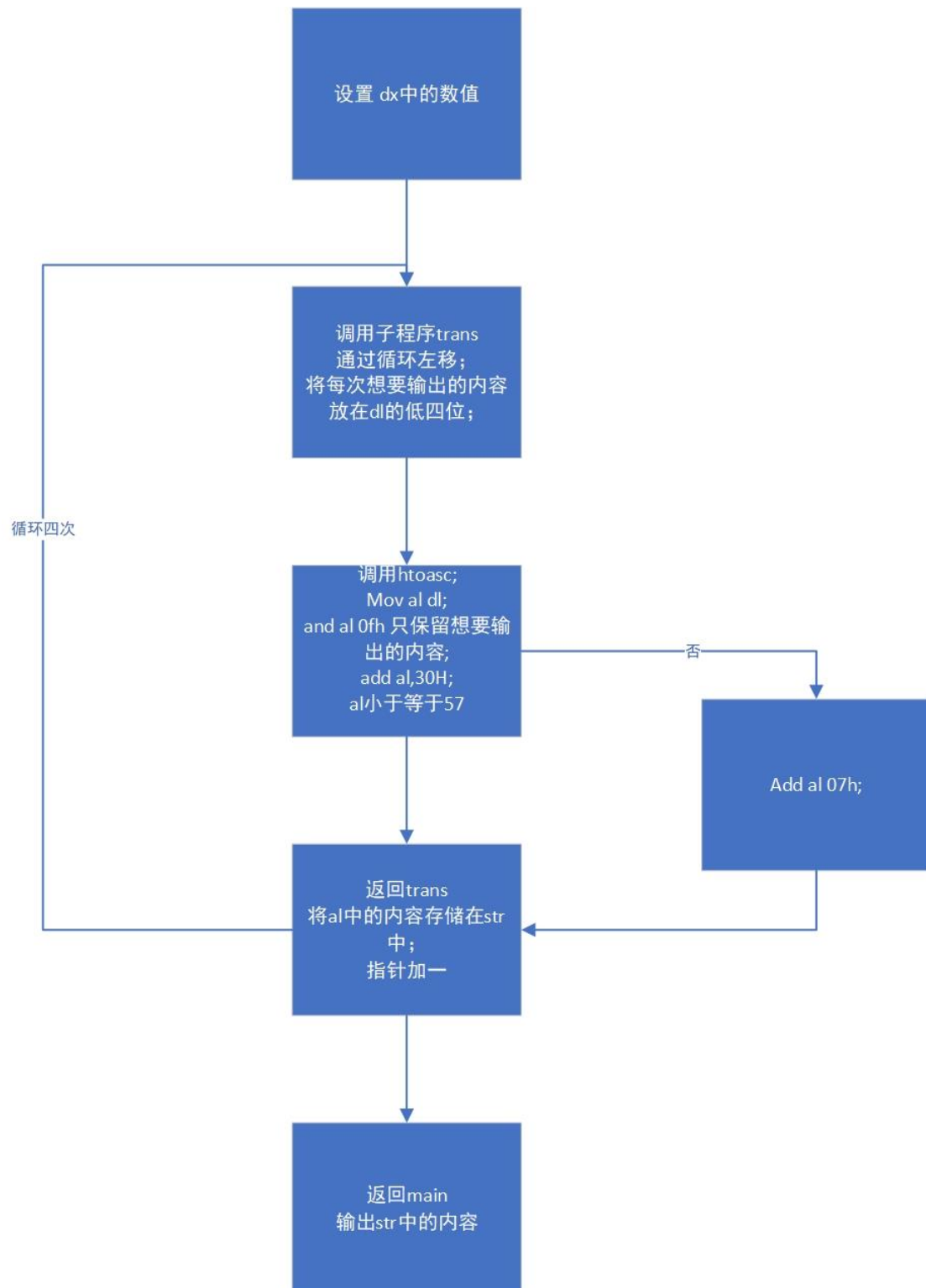
1. 继续掌握汇编语言程序设计及调试方法
2. 初识子程序

二、实验思路，结果与分析

1. 阅读 sample7_1 的程序代码片段，并编程实现

程序介绍：以字符形式输出 dx 中的内容

流程图：



代码:

```
01 ; multi-segment executable file template.
02
03 data segment
04     ; add your data here!
05     str 4 dup(0), 10, 13, '$'
06 ends
07
08 code segment
09 start:
10 ; set segment registers:
11     mov ax, data
12     mov ds, ax
13     mov es, ax
14
15 main    proc    far
16     mov dx, 0A123H
17     lea bx, str           ; bx指针指向str，之后通过bx在str中存储字符
18     call trans
19
20     lea dx, str           ; 输出str中的内容，也就是dx中的内容的ascii形式
21     mov ah, 9
22     int 21h
23
24     mov ah, 1
25     int 21h
26
27     mov ax, 4c00h
28     int 21h
29 main    endp
30
31 trans proc near           ; trans程序的作用在于，将dx中的内容以字符形式存储在str中
32     mov cx, 4
33 trans1:
34     rol dx, 4             ; 每次循环左移4位，将想要输出的高位保存在最低四位
35
36     mov al, dl
37     call htoasc
38     mov [bx], al          ; 使用bx指针，在str中存储
39     inc bx                ; bx中一共有4*4个二进制位
40     loop trans1
41     ret
42 trans endp
43
44 htoasc proc near          ; htoasc的作用在于将al的最低四位的内容转换为ascii形式
45     and al, 0fh
46     add al, 30H
47     cmp al, 39H           ; 字母和数字的处理不同
48     jbe exit
49     add al, 7
50 exit:
51     ret
52 htoasc endp
53 ends
54 end start ; set entry point and stop the assembler.
55
```

结果：



2. 重新完成实验 6 第 3 题、第 4 题（实验六已完成且结果正确的可以不做）

当时做了两题，且都正确且时间复杂度不高

3. 计算 $1! + 2! + \dots + 8!$ 的值

要求： 计算结果保存到以 `result` 开始的内存地址中

尝试将结果以 10 进制方式输出到屏幕中（选作）

尝试用子程序方式解决（选作）

提示： 需要解决溢出问题

流程图：



代码:

```
01 ; multi-segment executable file template.
02
03 data segment
04     ; add your data here!
05     res db 5 dup(0)
06     k dw 10
07 ends
08
09 code segment
10 start:
11 ; set segment registers:
12     mov ax, data
13     mov ds, ax
14     mov es, ax
15
16 main PROC FAR
17     mov cx, 8 ; 此处循环系数正好也是8, 通过此循环, 求得8! + 7! + 6! + ... + 1
18 loop1:
19     mov ax, cx ;
20     call calcul
21     add word ptr res, ax ; 将ax中的结果保存在res
22     adc (res+2), 0 ; 如果进位则存储在这里, 实际上并没有
23     loop loop1
24
25     mov ax, word ptr res
26     call convert ; 转换数字形式
27
28     dec di
29     call prin ; 倒序输出res中的内容
30
31     mov ah, 1
32     int 21h
33
34     mov ax, 4c00h ; exit to operating system.
35     int 21h
```



```

35     int 21h
36 main endp
37
38
39 ;倒序输出res中的内容
40 prin proc near
41
42 loop4:
43     mov dl, res[di]
44     mov ah, 02h           ;配合指针的移动，和02号中断，逐个输出
45     int 21h
46     cmp di, 0
47     jz exit3
48     dec di               ;指针从后往前
49     jmp loop4
50 exit3:
51     ret
52 prin endp
53
54 ;转换，将ax中的数字转换为十进制形式，以字符形式倒序存储在首地址为res的数组里
55 convert proc near
56     mov di, 0
57 loop2:
58     cmp ax, 0
59     jz exit2
60     div k
61     add dl, 48           ;转换为字符形式
62     mov res[di], dl      ;存储在相应位置
63     inc di               ;指针加一，存在下一个位置
64     mov dl, 0
65     jmp loop2
66 exit2:
67     ret
68 convert endp

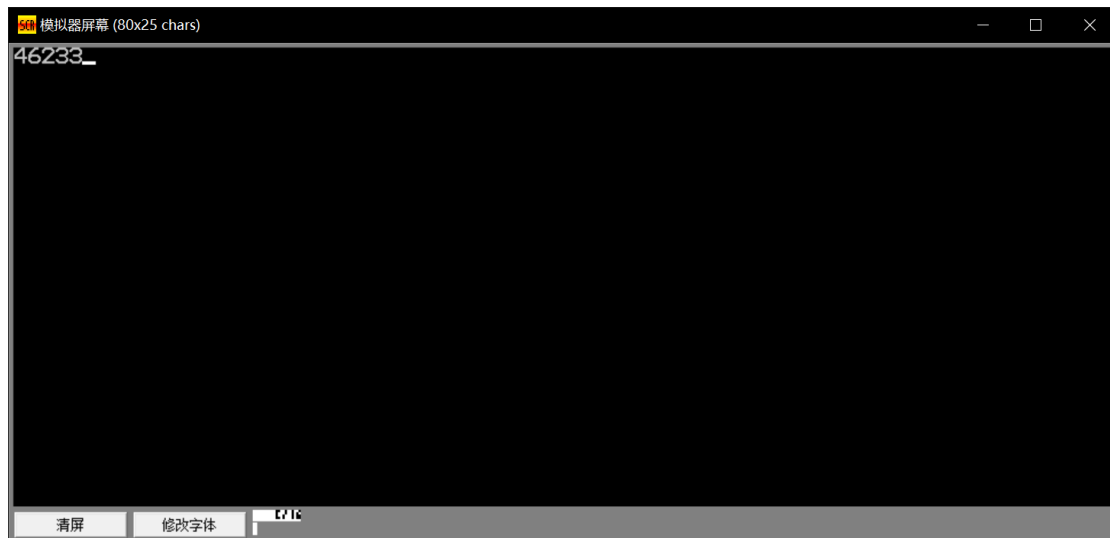
```

```

69
70 ;累乘循环 该模块负责的是将ax中的内容求阶乘并保留在ax中
71 calcu proc near
72     mov bx, ax
73 loop3:
74     dec bx               ;bx初始值为[ax]-1, 之后每次减一
75     cmp bx, 0
76     jz exit
77     mul bx
78     jmp loop3
79 exit:
80     ret
81 calcu endp
82
83 ends
84
85 end start ; set entry point and stop the assembler.

```

运行结果：



四、实验心得

对于 子程序的封装有了初步的认识

学会了 写简单的子程序调用

对于 以前所有有了更深的认识和体会

问题：

指令的记忆，需要多次查阅